# Self-Evolving Agent Optimization with Multi-Provider Coding Agents: A Study on AppWorld with auto-agent and Kiro CLI

Fabio Nonato de Paula
nonatofabio@github

March 2026

**Abstract**

We present an empirical study of **auto-agent**, a self-evolving agent optimization system that autonomously improves AI agents through iterative, hypothesis-driven code modifications. We extend auto-agent with a provider abstraction supporting multiple coding agent backends, including a new integration with Kiro CLI alongside the original Claude Code support. We evaluate the system on AppWorld, a benchmark of 732 interactive coding tasks across 9 simulated real-world applications. Using Kiro CLI as the orchestrating coding agent, we measure how effectively auto-agent improves a minimal AppWorld agent across five Claude models on all 90 train tasks. Starting from 0% accuracy, auto-agent achieves 93.3% for Opus 4.6 and 90.0% for Sonnet 4.6 in just two accepted iterations, while smaller models reach 31–43%. An orchestrator ablation reveals that a Sonnet-tier orchestrator outperforms an Opus orchestrator for the Sonnet 4.6 target (90% vs 78%), challenging the assumption that stronger orchestrators are always preferable. These results demonstrate that autonomous agent improvement is feasible at scale, with the target model's capability as the dominant factor and a surprising nonlinear scaling relationship.

## 1 Introduction

The development of AI agents that interact with external tools and APIs is a rapidly growing area, yet improving these agents remains largely a manual process. Developers analyze failure cases, hypothesize improvements, implement changes, and evaluate results—a cycle that is time-consuming and difficult to scale.

**auto-agent**[1] addresses this by automating the optimization loop. Given a target agent repository and a golden dataset of expected behaviors, auto-agent spawns a coding agent to analyze failures, implement fixes, run evaluations, and decide whether to accept or rollback changes. Each iteration produces a hypothesis—a single, scoped improvement attempt—with accumulated learnings persisted across iterations.

In this work, we make three contributions:

1. **Provider abstraction**: We decouple auto-agent from Claude Code, introducing a pluggable provider interface that supports both Claude Code and Kiro CLI as coding agent backends.

2. **Kiro CLI integration**: We implement a Kiro CLI provider that uses steering files for system prompt injection while preserving full tool access, resolving a limitation where Kiro's `-agent` flag disables built-in tools.

[1] `https://github.com/alfonsograziano/auto-agent`

3. **AppWorld evaluation**: We evaluate auto-agent's optimization capability on AppWorld, a challenging benchmark of interactive coding tasks, across four Claude model tiers serving as the target agent's LLM backbone.

# 2 Background

## 2.1 auto-agent

auto-agent uses a two-repository architecture: an *orchestrator* that controls the optimization loop, and a *target agent* repository containing the agent being improved. The orchestrator spawns a coding agent inside the target repository with full context about the job objective, prior learnings, and evaluation results.

Each optimization iteration follows a fixed protocol:

1. Read the baseline report, accumulated memory (`MEMORY.md`), and job configuration
2. Create a git branch for the new hypothesis
3. Spawn the coding agent with full context
4. The coding agent analyzes failures, implements a fix targeting one failure class, and runs evaluations
5. Parse the decision (`CONTINUE` or `ROLLBACK`) from the structured report
6. Accept or discard the hypothesis branch accordingly

A shared `MEMORY.md` file persists learnings across iterations, preventing the system from repeating failed strategies.

## 2.2 AppWorld

AppWorld [1] is a benchmark of 732 autonomous agent tasks built on a simulated world of 9 real-world applications (Spotify, Amazon, Venmo, etc.) with 457 APIs and approximately 100 simulated users. Tasks require interactive Python code generation to call APIs, navigate paginated results, and complete multi-step workflows. Evaluation is state-based: database unit tests verify whether the agent achieved the correct outcome regardless of the code path taken.

AppWorld is particularly challenging because agents must:

- Discover correct API signatures through documentation
- Handle authentication flows (login, access tokens)
- Navigate paginated API responses
- Compose multi-step workflows across applications

## 2.3 Kiro CLI

Kiro CLI[2] is a terminal-based AI coding assistant. Unlike Claude Code, which provides `-system-prompt` and `-add-dir` flags for programmatic control, Kiro CLI requires alternative integration strategies. We discovered that Kiro's `-agent` flag disables all built-in tools (file read/write, shell execution), making it unsuitable for auto-agent's use case. Instead, we use *steering files*—markdown files placed in `.kiro/steering/` with `inclusion: always` frontmatter—which inject system prompts while preserving full tool access.

[2] `https://kiro.dev/cli/`

# 3 System Design

## 3.1 Provider Abstraction

We introduce an `AgentProvider` interface with three methods:

```
interface AgentProvider {
  readonly name: string;
  assertInstalled(): void;
  run(opts: AgentRunOptions): Promise<number>;
  cleanup?(): Promise<void>;
}
```

The `ClaudeProvider` wraps Claude Code's CLI with `-system-prompt` and `-add-dir` flags. The `KiroProvider` writes a temporary steering file to `.kiro/steering/` and folds the job directory context into the user prompt. Provider selection is configured via a `**Provider**` field in the job's `JOB.md`.

## 3.2 AppWorld Target Agent

We construct a minimal target agent consisting of three files:

- `agent.py`: Contains the system prompt, LLM calling logic, and a `solve_task()` function implementing a ReAct-style loop
- `llm_call.py`: A subprocess-based LLM caller that invokes Amazon Bedrock outside of AppWorld's `freezegun` time patching (which otherwise breaks AWS SigV4 signature validation)
- `run_eval.py`: An evaluation runner that iterates over AppWorld tasks and outputs JSON results

The agent starts with a deliberately minimal system prompt and no API exploration logic, establishing a 0% baseline that auto-agent must improve.

# 4 Experimental Setup

## 4.1 Models

We evaluate six Claude models available via Amazon Bedrock, spanning three capability tiers:

| **Model** | **Tier** | **Generation** |
|---|---|---|
| Claude 3.5 Haiku | Haiku (small) | Previous |
| Claude Haiku 4.5 | Haiku (small) | Current |
| Claude Sonnet 4 | Sonnet (medium) | Previous |
| Claude Sonnet 4.5 | Sonnet (medium) | Current |
| Claude Sonnet 4.6 | Sonnet (medium) | Latest |
| Claude Opus 4.6 | Opus (large) | Latest |

Table 1: Target agent models evaluated. All accessed via Bedrock cross-region inference profiles in `us-east-1`.

### 4.2 Configuration

All experiments use:

- **Orchestrating agent**: Kiro CLI (the coding agent that modifies the target agent)
- **Dataset**: 10 AppWorld `train` tasks (Spotify queries, mutations, multi-app workflows)
- **Max iterations**: 3 per model
- **Forbidden files**: `run_eval.py`, `llm_call.py`, `data/`
- **Modifiable**: `agent.py` (system prompt, agent loop logic, helper functions)

The orchestrating Kiro agent receives the same job configuration for all four experiments, ensuring that differences in outcomes are attributable to the target model's capability rather than prompt variation.

## 5 Results

We report results from full-scale experiments on all 90 AppWorld train tasks with 5-way parallel evaluation. All baselines were validated to contain real LLM work (18–27 cases with multi-pass test results per experiment). All CONTINUE iteration accuracies were cross-checked against raw `passed_cases` values in eval JSON output captured in optimization logs.

### 5.1 Quantitative Results

| Model | Base | Iter 1 | Iter 2 | Iter 3 | Final | Δ |
|---|---|---|---|---|---|---|
| Haiku 4.5 | 0.0% | 27.8% (C) | 27.8% (R) | 31.1% (C) | 31.1% | +31.1 |
| Sonnet 4 | 0.0% | 8.9% (C) | 22.2% (C) | 40.0% (C) | 40.0% | +40.0 |
| Sonnet 4.5 | 0.0% | 42.2% (C) | 43.3% (C) | 43.3% (R) | 43.3% | +43.3 |
| Sonnet 4.6 | 0.0% | 84.4% (C) | **90.0%** (C) | 83.3% (R) | **90.0%** | +90.0 |
| Opus 4.6 | 4.4% | 83.3% (C) | **93.3%** (C) | 92.2% (R) | **93.3%** | +88.9 |

Table 2: Accuracy on 90 AppWorld train tasks across optimization iterations. (C) = accepted, (R) = rolled back. Kiro CLI in auto mode (Sonnet-tier) as orchestrator. All results validated against raw eval JSON.

### 5.2 Qualitative Analysis

**Model capability determines the optimization ceiling.** The same orchestrator making similar structural improvements produces 31–93% final accuracy depending on the target model. Sonnet 4.6 and Opus 4.6 both exceed 90%, while Haiku 4.5 plateaus at 31%.

**Iteration dynamics vary by model tier.** Sonnet 4 is the only model to improve monotonically across all three iterations (8.9% → 22.2% → 40.0%, all CONTINUE). Sonnet 4.6 and Opus 4.6 achieve their best results by iteration 2, with iteration 3 causing slight regressions that are correctly rolled back. Haiku 4.5 shows a non-monotonic pattern where iteration 2 regresses before iteration 3 recovers.

**Structural improvements are consistent.** Across all five experiments, the Kiro orchestrator made similar first-iteration changes: adding an API exploration step (`_gather_context`), fixing the system prompt with correct API signatures and authentication patterns, increasing `MAX_STEPS`, and expanding `maxTokens`. Later iterations addressed pagination handling, data structure documentation, and edge case formatting rules.

**Opus 4.6 has a non-zero baseline.** It is the only model that solves any tasks ($4/90 = 4.4\%$) even with the bare-bones agent, demonstrating that frontier models can partially compensate for poor scaffolding through raw capability.

**Iteration 3 frequently regresses.** Four of five main experiments had iteration 3 rolled back, suggesting diminishing returns after two iterations of prompt and scaffolding improvements. The remaining failures likely require fundamentally different strategies (e.g., adding new tools, multi-agent architectures) rather than incremental prompt refinement.

### 5.3 Orchestrator Ablation

| Target | Orchestrator | Iter 1 | Iter 2 | Iter 3 | Final |
|---|---|---|---|---|---|
| Haiku 4.5 | Auto (Sonnet) | 27.8% (C) | 27.8% (R) | 31.1% (C) | 31.1% |
| Haiku 4.5 | Opus 4.6 | 8.9% (C) | 31.1% (C) | 28.9% (R) | 31.1% |
| Sonnet 4.6 | Auto (Sonnet) | 84.4% (C) | **90.0%** (C) | 83.3% (R) | **90.0%** |
| Sonnet 4.6 | Opus 4.6 | 55.6% (C) | 74.4% (C) | 77.8% (C) | 77.8% |

Table 3: Ablation: same target model, different orchestrator. The auto (Sonnet-tier) orchestrator outperforms Opus 4.6 for Sonnet 4.6 (90.0% vs 77.8%), while both reach 31.1% for Haiku 4.5.

The ablation reveals a surprising result: the Opus 4.6 orchestrator does *not* outperform the auto (Sonnet-tier) orchestrator. For Sonnet 4.6, the auto orchestrator achieves 90.0% vs. Opus's 77.8%. For Haiku 4.5, both converge to 31.1% via different trajectories.

This suggests that orchestrator–target capability matching is not simply "bigger is better." The Sonnet-tier orchestrator may generate changes that are better calibrated to what the target model can execute, while the Opus orchestrator's more sophisticated modifications may introduce complexity that doesn't translate to accuracy gains. The Opus orchestrator also takes a more gradual path (55.6% $\rightarrow$ 74.4% $\rightarrow$ 77.8% for Sonnet 4.6), suggesting it makes more conservative, incremental changes.

## 6 Discussion

### 6.1 Scaling Behavior

The relationship between model capability and optimization ceiling is strongly nonlinear. Haiku 4.5 (31%) and Sonnet 4 (40%) show modest gains, while Sonnet 4.6 (90%) and Opus 4.6 (93%) achieve near-ceiling performance. The jump from Sonnet 4.5 (43%) to Sonnet 4.6 (90%) is particularly striking—a single generation upgrade in the target model doubles the optimization ceiling, suggesting that auto-agent's effectiveness scales superlinearly with model capability.

### 6.2 Practical Implications

For practitioners, these results suggest:

1. **Use the best target model you can afford.** The optimization ceiling is primarily determined by the target model, not the orchestrator. Sonnet 4.6 at 90% is a better investment than Opus orchestrator + weaker target.

2. **Two iterations suffice.** Most gains are captured by iteration 2. Iteration 3 frequently regresses, suggesting a natural stopping point.

3. **The orchestrator doesn't need to be frontier-class.** A Sonnet-tier orchestrator matches or exceeds Opus for the experiments we tested, at lower cost.

### 6.3 Limitations

- **Single benchmark**: We evaluated only on AppWorld train tasks. Results may differ on other benchmarks or task types.
- **Three iterations**: More iterations might yield further gains for models that haven't plateaued (e.g., Sonnet 4).
- **Non-determinism**: LLM-based agents are inherently non-deterministic. Results may vary across runs. We did not run multiple seeds.
- **Concurrency artifacts**: Parallel evaluation (5-way) caused occasional `freezegun` and SQLite conflicts in AppWorld, potentially affecting a small number of task results.

## 7 Related Work

**Autoresearch** [2] automates the research process by having LLMs generate hypotheses, run experiments, and iterate. auto-agent applies a similar philosophy to agent development.

**DSPy** [3] optimizes LLM pipelines by tuning prompts and few-shot examples. Unlike auto-agent, DSPy operates at the prompt level rather than modifying agent code.

**AppWorld** [1] provides the benchmark used in our evaluation. The official leaderboard shows that even state-of-the-art models achieve modest scores on the challenge set, confirming the benchmark's difficulty.

## 8 Conclusion

We demonstrated that auto-agent, extended with Kiro CLI support, can autonomously improve an AppWorld agent from 0% to 93.3% accuracy for Opus 4.6 and to 90.0% for Sonnet 4.6 over just two accepted optimization iterations on 90 tasks. The provider abstraction enables fair comparison across coding agent backends, and the AppWorld benchmark provides a rigorous evaluation framework.

Three findings emerge. First, the target model's capability is the dominant factor: the same orchestrator produces 31–93% accuracy depending on the target model, with a sharp nonlinear jump between Sonnet 4.5 (43%) and Sonnet 4.6 (90%). Second, two iterations capture most gains—iteration 3 was rolled back in four of five experiments, suggesting natural diminishing returns from prompt and scaffolding improvements. Third, a stronger orchestrator does not guarantee better results: the Sonnet-tier orchestrator outperformed Opus 4.6 as orchestrator for the Sonnet 4.6 target (90% vs 78%), challenging the assumption that more capable orchestrators are always preferable.

Future work should explore more iterations for models still improving (Sonnet 4), alternative improvement strategies beyond prompt engineering (tool creation, multi-agent architectures), multiple random seeds for statistical robustness, and evaluation on AppWorld's held-out test sets.

## References


[1] Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. AppWorld: A Controllable World of Apps and People for Benchmarking Interactive Coding Agents. In *ACL*, 2024.

[2] Andrej Karpathy. Autoresearch. `https://github.com/karpathy/autoresearch`, 2025.

[3] Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. In *ICLR*, 2024.